DAA – Group A Practicals

Simple Java programs with comments, terminal inputs, outputs and short explanations

Included: Fibonacci (Non-Recursive & Recursive), Job Sequencing, Huffman Coding, Fractional Knapsack and 0/1 Knapsack.

Practical 1A – Fibonacci Series (Non-Recursive)

Aim / Concept: Generate the Fibonacci series using a loop. Each next number is the sum of the previous two numbers.

Java code (save using this file name):

```
import java.util.Scanner;

public class FibonacciNonRecursive {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Take number of terms from user
        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();

        // First two Fibonacci numbers
        int a = 0, b = 1, c;

        // Repeat n times
        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");

            // Calculate next number
            c = a + b;

            // Update values for next iteration
            a = b;
            b = c;
        }

        sc.close();
    }
}
```

Terminal input and expected output:

```
Input: 7
Output: 0 1 1 2 3 5 8
```

How to explain: Explain: The for loop prints the current value of a. Then c = a + b calculates the next number, and a and b are updated.

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Practical 1B – Fibonacci Series (Recursive)

Aim / Concept: Generate Fibonacci numbers using a function that calls itself.

Java code (save using this file name):

```
import java.util.Scanner;

public class FibonacciRecursive {

    // Function returns the nth Fibonacci number
    static int fib(int n) {
        // Base conditions stop recursion
        if (n == 0) return 0;
        if (n == 1) return 1;

        // Add the previous two Fibonacci numbers
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Take number of terms
        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();

        // Print each Fibonacci number
        for (int i = 0; i < n; i++) {
            System.out.print(fib(i) + " ");
```

```
        }

        sc.close();
    }
}
```

Terminal input and expected output:

```
Input: 7
Output: 0 1 1 2 3 5 8
```

How to explain: Explain: fib() calls itself to calculate fib(n-1) + fib(n-2). The base conditions for 0 and 1 stop the recursive calls.

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Practical 2 – Job Sequencing with Deadlines

Aim / Concept: Arrange jobs to get maximum profit. Each job takes one time slot. Jobs are considered in decreasing order of profit.

Java code (save using this file name):

```
import java.util.Scanner;

public class JobSequencing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of jobs: ");
        int n = sc.nextInt();

        char[] id = new char[n];
        int[] deadline = new int[n];
        int[] profit = new int[n];

        // Read job ID, deadline and profit
        for (int i = 0; i < n; i++) {
            System.out.print("Enter job ID, deadline and profit: ");
            id[i] = sc.next().charAt(0);
            deadline[i] = sc.nextInt();
            profit[i] = sc.nextInt();
        }

        // Sort jobs by profit in descending order
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (profit[i] < profit[j]) {
                    int temp = profit[i];
                    profit[i] = profit[j];
                    profit[j] = temp;

                    temp = deadline[i];
                    deadline[i] = deadline[j];
                    deadline[j] = temp;

                    char tempId = id[i];
                    id[i] = id[j];
                    id[j] = tempId;
                }
            }
        }

        // Find the largest deadline
        int max = 0;
        for (int i = 0; i < n; i++) {
            if (deadline[i] > max) max = deadline[i];
        }

        // Create empty time slots
        char[] slot = new char[max];
        for (int i = 0; i < max; i++) slot[i] = '-';

        int totalProfit = 0;

        // Place each job in the latest free slot
        for (int i = 0; i < n; i++) {
            for (int j = deadline[i] - 1; j >= 0; j--) {
                if (slot[j] == '-') {
                    slot[j] = id[i];
                    totalProfit += profit[i];
                    break;
                }
            }
        }

        System.out.print("Selected Jobs: ");
```

```
        for (int i = 0; i < max; i++) {
            if (slot[i] != '-') System.out.print(slot[i] + " ");
        }

        System.out.println("\nMaximum Profit: " + totalProfit);
        sc.close();
    }
}
```

Terminal input and expected output:

```
Input:
5
A 2 100
B 1 19
C 2 27
D 1 25
E 3 15

Output:
Selected Jobs: C A E
Maximum Profit: 142
```

How to explain: Explain: Sort jobs by profit. For each job, search backward from its deadline and put it in the latest empty slot. Add its profit when scheduled.

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Practical 3 – Huffman Coding

Aim / Concept: Build a Huffman tree by repeatedly combining the two nodes with the smallest frequencies. Left edge is 0 and right edge is 1.

Java code (save using this file name):

```
import java.util.PriorityQueue;
import java.util.Scanner;

class Node {
    char ch;
    int freq;
    Node left, right;

    // Create a node with a character and frequency
    Node(char ch, int freq) {
        this.ch = ch;
        this.freq = freq;
    }
}

public class HuffmanCoding {

    // Traverse the tree and print each character's code
    static void printCode(Node root, String code) {
        if (root.left == null && root.right == null) {
            System.out.println(root.ch + " : " + code);
            return;
        }

        // Left edge adds 0
        printCode(root.left, code + "0");

        // Right edge adds 1
        printCode(root.right, code + "1");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Priority queue keeps the smallest frequency first
        PriorityQueue<Node> pq = new PriorityQueue<>(
            (a, b) -> a.freq - b.freq
        );

        System.out.print("Enter number of characters: ");
        int n = sc.nextInt();

        // Read each character and its frequency
        for (int i = 0; i < n; i++) {
            System.out.print("Enter character and frequency: ");
            char ch = sc.next().charAt(0);
            int freq = sc.nextInt();
            pq.add(new Node(ch, freq));
        }

        // Combine the two minimum-frequency nodes
        while (pq.size() > 1) {
```

```
            Node left = pq.poll();
            Node right = pq.poll();

            Node parent = new Node('-', left.freq + right.freq);
            parent.left = left;
            parent.right = right;

            pq.add(parent);
        }

        // The last remaining node is the tree root
        Node root = pq.poll();

        System.out.println("Huffman Codes:");
        printCode(root, "");

        sc.close();
    }
}
```

Terminal input and expected output:

```
Input:
4
A 5
B 9
C 12
D 13

One possible output:
C : 00
D : 01
A : 100
B : 101
```

How to explain: Explain: A priority queue selects the two smallest frequencies. They are joined under a new parent node. This repeats until one tree remains. Different valid codes may appear if equal frequencies are ordered differently.

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Practical 4 – Fractional Knapsack

Aim / Concept: Choose items by highest profit-to-weight ratio. A fraction of the final item can be taken.

Java code (save using this file name):

```
import java.util.Scanner;

public class FractionalKnapsack {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of items: ");
        int n = sc.nextInt();

        int[] weight = new int[n];
        int[] profit = new int[n];
        double[] ratio = new double[n];

        // Read weights and profits and calculate ratios
        for (int i = 0; i < n; i++) {
            System.out.print("Enter weight and profit: ");
            weight[i] = sc.nextInt();
            profit[i] = sc.nextInt();
            ratio[i] = (double) profit[i] / weight[i];
        }

        System.out.print("Enter knapsack capacity: ");
        int capacity = sc.nextInt();

        // Sort items by ratio from highest to lowest
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (ratio[i] < ratio[j]) {
                    double tr = ratio[i];
                    ratio[i] = ratio[j];
                    ratio[j] = tr;

                    int tw = weight[i];
                    weight[i] = weight[j];
                    weight[j] = tw;

                    int tp = profit[i];
                    profit[i] = profit[j];
                    profit[j] = tp;
                }
            }
        }
```

```
        double totalProfit = 0;

        // Fill the bag using the best ratio first
        for (int i = 0; i < n; i++) {
            if (weight[i] <= capacity) {
                // Take the complete item
                totalProfit += profit[i];
                capacity -= weight[i];
            } else {
                // Take only the fraction that fits
                double fraction = (double) capacity / weight[i];
                totalProfit += profit[i] * fraction;
                capacity = 0;
                break;
            }
        }

        System.out.println("Maximum Profit: " + totalProfit);
        sc.close();
    }
}
```

Terminal input and expected output:

```
Input:
3
10 60
20 100
30 120
50

Output:
Maximum Profit: 240.0
```

How to explain: Explain: Calculate profit/weight for every item, sort by this ratio, then take complete items while possible. If the next item does not fit, take only the fraction that fits.

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Practical 5 – 0/1 Knapsack

Aim / Concept: Use Dynamic Programming. Each item is either fully selected or rejected; fractions are not allowed.

Java code (save using this file name):

```
import java.util.Scanner;

public class Knapsack01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of items: ");
        int n = sc.nextInt();

        int[] weight = new int[n];
        int[] profit = new int[n];

        // Read weight and profit of each item
        for (int i = 0; i < n; i++) {
            System.out.print("Enter weight and profit: ");
            weight[i] = sc.nextInt();
            profit[i] = sc.nextInt();
        }

        System.out.print("Enter knapsack capacity: ");
        int W = sc.nextInt();

        // dp[i][w] stores best profit using first i items
        // with bag capacity w
        int[][] dp = new int[n + 1][W + 1];

        // Consider items one by one
        for (int i = 1; i <= n; i++) {

            // Check all capacities from 0 to W
            for (int w = 0; w <= W; w++) {

                if (weight[i - 1] <= w) {
                    // Profit if we include the current item
                    int include = profit[i - 1]
                        + dp[i - 1][w - weight[i - 1]];

                    // Profit if we exclude the current item
                    int exclude = dp[i - 1][w];

                    // Store the better choice
                    if (include > exclude) {
                        dp[i][w] = include;
```

```
                    } else {
                        dp[i][w] = exclude;
                    }
                } else {
                    // Item is too heavy, so exclude it
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }

        // Last cell contains the maximum profit
        System.out.println("Maximum Profit: " + dp[n][W]);

        sc.close();
    }
}
```

Terminal input and expected output:

```
Input:
3
10 60
20 100
30 120
50

Output:
Maximum Profit: 220
```

How to explain: Explain: The DP table stores the best profit for each number of items and each capacity. For every item, compare including it and excluding it. The final answer is dp[n][W].

Run in terminal: javac FileName.java then java ClassName. Use the same spelling as the file/class name.

Quick revision

Fibonacci: iteration / recursion. **Job Sequencing:** Greedy. **Huffman:** Greedy + binary tree. **Fractional Knapsack:** Greedy + ratio. **0/1 Knapsack:** Dynamic Programming.

Note: These are educational sample programs. For practical submission, match the exact input format and required method from your college's lab manual.